Technical Report: Book Recommendation API

Module: XJCO3011 Web Services and Web Data **Assignment:** Coursework 1 - Individual Web Services API Development Project

1. Introduction and Project Overview

This report outlines the design, development, and implementation of a Book Recommendation API. The primary objective of this project is to provide a robust, data-driven web service that manages book metadata and user reviews while offering analytical endpoints such as genre trends and rating distributions. The API adheres to RESTful architectural principles, providing full Create, Read, Update, and Delete (CRUD) capabilities across multiple interconnected data models. The dataset utilized for this project is derived from the publicly available Goodreads dataset on Kaggle, which provides realistic and comprehensive book information.

2. Technology Stack Justification

The technology stack was carefully selected to balance rapid development, maintainability, and scalability, aligning with modern software engineering practices.

2.1 Backend Framework: Django & Django REST Framework (DRF)

Django was chosen as the core web framework due to its “batteries-included” philosophy, which significantly accelerates development. Its built-in Object-Relational Mapping (ORM) system simplifies database interactions and schema migrations, allowing for a focus on business logic rather than low-level SQL queries. Django REST Framework (DRF) was integrated to extend Django’s capabilities for building Web APIs. DRF provides powerful serialization, built-in pagination, and filtering mechanisms out of the box, which are essential for handling large datasets efficiently.

2.2 Database: SQLite

For the scope of this coursework and local development, SQLite was selected as the database engine. It requires zero configuration and stores the entire database as a single file, making it highly portable and easy to manage during the development and testing phases. While not suitable for high-concurrency production environments, it perfectly meets the requirements of this academic project.

2.3 API Documentation: drf-spectacular

To fulfill the requirement for comprehensive API documentation, `drf-spectacular` was utilized. This tool automatically generates OpenAPI 3.0 schemas from the DRF codebase, enabling the seamless creation of interactive Swagger UI documentation. This approach ensures that the documentation is always synchronized with the actual codebase, reducing maintenance overhead and improving accuracy.

3. Design and Architecture

The API is structured around two primary data models: `Book` and `Review`.

3.1 Data Models

- **Book Model:** Stores metadata such as ISBN, title, author, publication year, publisher, and aggregated statistics (average rating and ratings count).
- **Review Model:** Represents user feedback, containing the user's name, a rating (1-5), a text comment, and a timestamp.

These models are linked via a one-to-many relationship (one Book can have multiple Reviews). A key architectural decision was to implement a method within the `Book` model to automatically recalculate and update its `average_rating` and `ratings_count` whenever a related `Review` is created, updated, or deleted. This denormalization strategy optimizes read performance for the `/books/` endpoint, as the aggregated statistics do not need to be calculated on-the-fly for every request.

3.2 Endpoints and Features

The API exposes 13 distinct endpoints, exceeding the minimum requirement of four. Key features include:

- **Core CRUD:** Complete management of Books and Reviews.
- **Advanced Filtering & Search:** Utilizing `django-filter`, users can search by title/author and filter by publication year.
- **Analytics:** Custom actions were implemented to provide aggregated data, such as `/api/books/top-rated/` and `/api/books/author-stats/`, demonstrating the ability to perform complex database queries and data aggregation.

4. Challenges and Testing Approach

4.1 Challenges

A significant challenge encountered was ensuring the synchronization of aggregated data (average ratings) between the `Book` and `Review` models. Initially, the logic was placed within the view layer, which proved fragile. The solution was to encapsulate this logic within the `perform_create`, `perform_update`, and `perform_destroy` methods of the `ReviewViewSet`, ensuring that any modification to a review reliably triggers an update to the parent book's statistics.

Another challenge was properly configuring the DRF serializers to handle nested relationships, specifically allowing the `BookSerializer` to display associated reviews while preventing recursive serialization issues.

4.2 Testing Approach

Testing was conducted primarily through manual endpoint verification using the generated Swagger UI and browser-based DRF browsable API. Each endpoint was systematically tested for expected behavior, including edge cases such as submitting invalid data (e.g., ratings outside the 1-5 range) to verify that appropriate HTTP 400 Bad Request responses were returned. The data import script was also tested extensively to ensure it correctly handled missing or malformed rows in the Kaggle dataset without crashing.

5. Limitations and Future Improvements

Currently, the API lacks user authentication and authorization mechanisms. Anyone can modify or delete records. A crucial future improvement would be to implement token-based authentication (e.g., JWT) and permission classes to restrict write operations to registered users or administrators.

Additionally, the deployment is currently limited to local execution. Migrating the database to PostgreSQL and deploying the application to a cloud platform like PythonAnywhere or Heroku would enhance its real-world applicability and scalability.

6. Generative AI Declaration

In accordance with the "Green Light Assessment" policy, Generative AI tools were utilized throughout the development process.

Tools Used: Manus AI, GitHub Copilot.

Purposes:

1. **Planning and Architecture:** AI was used to brainstorm project ideas and structure the initial database schema.
2. **Debugging:** When encountering issues with DRF serializer routing (specifically attaching the correct serializer to custom `@action` methods), AI was used to identify the indentation error and provide the correct implementation for `get_serializer_class`.

Note: Exported conversation logs demonstrating this usage are available upon request or included as supplementary material in the project repository.